DELTA UNIVERSITY
FOR SCIENCE AND TECHNOLOGY

New Trends in Artificial Intelligence

AIE46

By

Dr Ahmed Zakzouk

Faculty of Artificial Intelligence

Unit 1: Introduction and General Trends

This unit lays the groundwork for the course: how AI evolved from rule-based systems to today's foundation models, and the major trends now shaping the field.

- **Evolution of AI: Expert Systems to Deep Learning:**

Rule-based expert systems dominated AI in the 1970s-80s, encoding human knowledge as explicit logical rules.

Statistical machine learning (1990s-2000s) introduced data-driven models such as SVMs and decision trees.

The deep learning revolution took off after 2012, driven by GPU compute and large labeled datasets (e.g., AlexNet).

Scaling laws linking data, compute, and model size are now the main driver behind foundation models.

- **Discriminative AI vs. Generative AI:**

• Discriminative models estimate P(y|x): they classify or predict a label given an input.

• Generative models estimate P(x) or P(x|z): they can create new, realistic content.

- Examples: logistic regression and CNN classifiers (discriminative) vs. GANs, VAEs, diffusion models, and LLMs (generative).
- Industry focus has shifted markedly toward generative approaches since 2020.

Foundation Models:

Defined as large models pretrained on broad data and adaptable to many downstream tasks. Term introduced by Stanford's Center for Research on Foundation Models (Bommasani et al., 2021).

Key characteristics: massive scale, emergent abilities, and a homogenization effect across applications. Examples include the GPT family, BERT, CLIP, and Stable Diffusion.

- **What is Discriminative AI?**

Core Goal: Discriminative AI learns the boundaries between different categories to classify or predict labels from input data.

How it Works: It looks at features in the data and decides which group or class they belong to. It does not try to create new data.

Common Uses:

1. Spam filtering (deciding if an email is spam or safe).
2. Fraud detection (flagging a fake bank transaction).
3. Medical diagnosis (identifying a tumor from a scan).
4. Sentiment analysis (reading a review to see if it is positive or negative).

Common Examples: Logistic regression, support vector machines (SVM), and standard convolutional neural networks (CNNs) used for recognition.

- **What is Generative AI?**

  Core Goal: Generative AI learns the full patterns and distribution of a dataset to synthesize brand-new, realistic outputs.

How it Works: Instead of drawing a line between categories, it understands how features relate to each other so it can build new text, images, audio, or code.

Common Uses:

1. Creating art, writing text, or composing music.
2. Generating synthetic data to train other systems.
3. Assisting with software coding.
4. Drug discovery (simulating new molecular structures).

Common Examples: Large Language Models (LLMs like GPT), Generative Adversarial Networks (GANs), and diffusion models

- **Foundation model:**

A **foundation model** is **a large-scale artificial intelligence system trained on vast, diverse datasets that serves as a versatile base layer to power many different specialized applications.**

Coined by researchers at Stanford HAI, these models use transfer learning and self-supervised training so they can adapt to a wide range of downstream tasks rather than just a single function.

Key Characteristics:

Massive Scale: Built using complex neural networks (like transformers) with billions or trillions of parameters.

Multimodal Capability: Can process and generate various data types simultaneously, including text, images, audio, video, and code.

Adaptability: Can be fine-tuned or prompted for specific use cases without needing to train a new model from scratch.

- **Examples:**

Large Language Models (LLMs): Text-focused models like OpenAI's GPT-4, Anthropic's Claude, and Meta's Llama.

Multimodal Models: Systems like Google's Gemini that handle text, vision, and audio concurrently.

Benefits & Challenges:

Benefits: Speeds up development time, reduces costs for specialized projects, and delivers powerful general reasoning skills.

Challenges: High computational training costs, environmental impact, potential for social bias, and data hallucinations.

Major Current Trends: Multimodal AI:

Core Definition: Systems that can simultaneously process, integrate, and reason across different data modalities like text, images, audio, video, and code.

How it Works: It uses unified embedding spaces and cross-modal encoders to combine diverse sensory inputs into a single contextual understanding.

Primary Focus: Perception and comprehension (giving the AI "senses" like sight, hearing, and reading).

Key Examples: Analyzing a video and an audio track together, or reading an image alongside a text prompt (e.g., GPT-4o or Gemini).

Best Used For: Rich content analysis, transcription, image/video recognition, and multi-format translation.

- **Agentic AI:**

  Core Definition: Systems equipped with agency, allowing them to autonomously plan tasks, use external tools, trigger APIs, and complete complex, multi-step workflows.

  How it Works: It uses reasoning loops (like Chain-of-Thought or ReAct), persistent memory, and orchestration frameworks (such as LangGraph or AutoGen) to decide the sequence of steps needed to reach an objective.

  Primary Focus: Action and execution (giving the AI hands to perform work and solve real-world problems).

  Key Examples: Monitoring inboxes, managing software deployment, executing research automation, or coordinating multi-step tasks across external databases.

  Best Used For: Workflow automation, complex problem-solving, operational tasks, and managing persistent software processes.

- **Multimodal and Agentic AI:**

  1. Multimodal AI processes and generates across text, image, audio, and video jointly.
  2. Agentic AI equips models with planning, memory, and tool use to act autonomously toward a goal.
  3. These two trends are converging in modern AI assistants and copilots.
  4. Both trends push AI from passive prediction toward active, embedded assistance.

  The AI Evolution Timeline:

  Expert Systems → Machine Learning → Deep Learning → Foundation Models → Generative & Agentic AI.

- **Case Study: From Classification to Creation:**
    1. A spam filter is a classic discriminative system: it labels an email as spam or not spam.
    2. A conversational assistant like a modern LLM is generative: it composes original, coherent text.
    3. The same underlying discipline now spans both paradigms, often within a single product.
    4. Understanding this distinction clarifies which approach fits a given problem.

- **Tools & Frameworks:**

1. scikit-learn — Classical machine learning algorithms and pipelines.
2. PyTorch / TensorFlow — Deep learning model development frameworks.
3. Hugging Face Transformers — Pretrained foundation models and fine-tuning utilities.
4. OpenAI / Anthropic APIs — Access to large-scale generative foundation models.

- **Challenges & Limitations:**

1. High data and compute costs required to train foundation models.
2. Homogenization risk: errors in one widely-reused model propagate across many applications.
3. Difficulty evaluating general-purpose models with traditional task-specific metrics.
4. Environmental impact of large-scale model training.

- **Unit 1 Summary**

- AI progressed from rule-based expert systems to data-driven and then deep learning approaches.
- Generative AI is redefining what AI systems can produce, not just predict.
- Foundation models are the technical backbone of most current AI trends.
- Multimodal and agentic capabilities define where the field is heading next.

Unit 2: Large Language Models (LLMs):

This unit examines the Transformer architecture and the training pipeline that turns raw text into capable, instructible language models. It surveys the major trends reshaping the field — scaling laws, retrieval augmentation, mixture-of-experts, and agentic tool use — and gives an in-depth, problem-and-solution treatment of the core challenges facing LLMs today: hallucination, bias, cost, limited context, security, evaluation, and reasoning.

1. Introduction: Why Large Language Models Matter:

Large Language Models (LLMs) are the defining trend of contemporary artificial intelligence. Built on the Transformer architecture and trained on web-scale text, models such as GPT, Claude, Gemini and LLaMA have moved language technology from narrow, task-specific systems toward general-purpose engines that can write, summarize, translate, reason, and act as the core of autonomous agents.

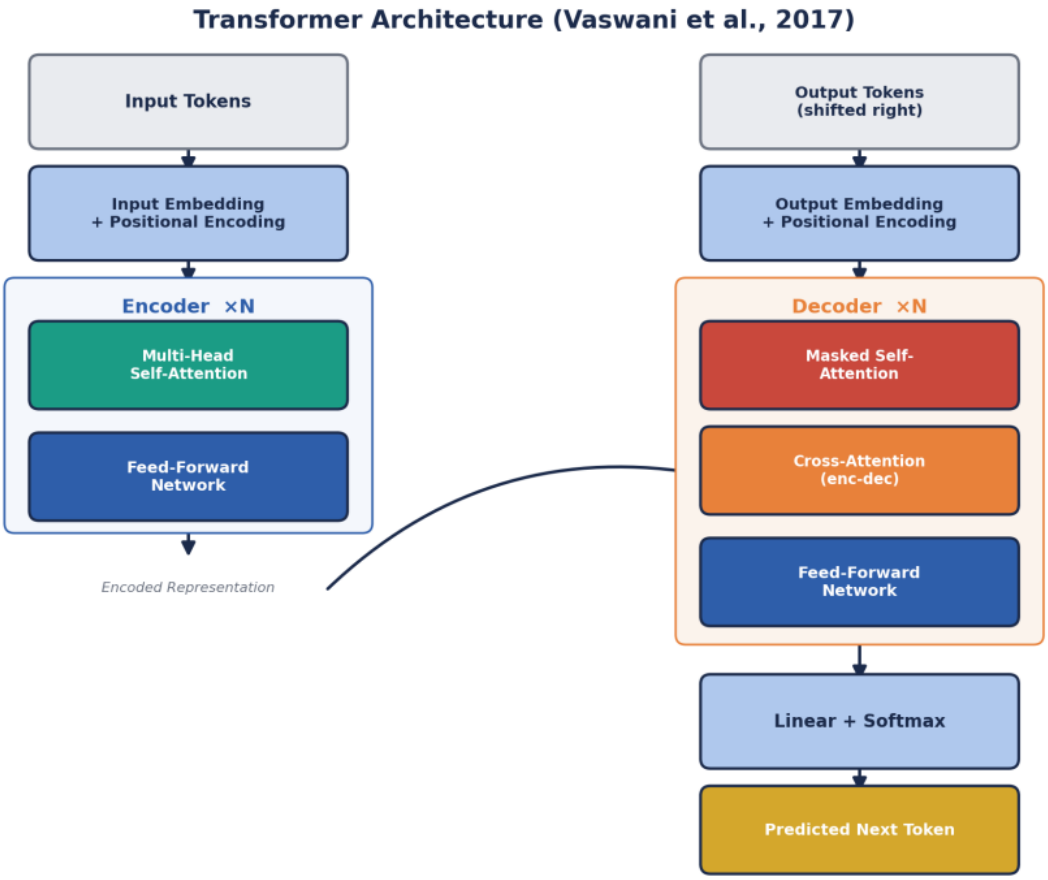
- **What Makes a Language Model "Large"?**

Scale operates along three axes that this unit returns to repeatedly: model size (parameters, from millions to over a trillion), data size (training tokens, now commonly in the trillions), and compute (FLOPs spent on training).

- **Large Language Models** are **deep neural networks — almost always Transformer-based — trained to predict the next token in a sequence over massive text corpora, then aligned to follow instructions and converse helpfully.**

2. The Transformer Architecture:

Every modern LLM is built on the Transformer, introduced in "Attention Is All You Need". Unlike earlier recurrent networks (RNNs, LSTMs) that process a sequence one token at a time, the Transformer processes all tokens in parallel and uses self-attention to let every token directly "look at “every other token, regardless of distance. This solved the long-range dependency and parallelization problems that limited RNNs.



igure 2.1 — The original encoder–decoder Transformer. Most modern LLMs (GPT, LLaMA, Claude) use a decoder-only variant: they keep the masked self-attention and feed-forward blocks on the right, dropping the separate encoder.

• Self-Attention: Query, Key, Value:

Self-attention lets each token build a representation that is a weighted combination of all other tokens' representations. Each token's embedding is projected into three vectors: a Query (what this token is looking for), a Key (what this token offers), and a Value (the content it contributes if attended to).

The attention weight between two tokens is the scaled dot product of one token's Query with another's Key, passed through a SoftMax so weights sum to 1.

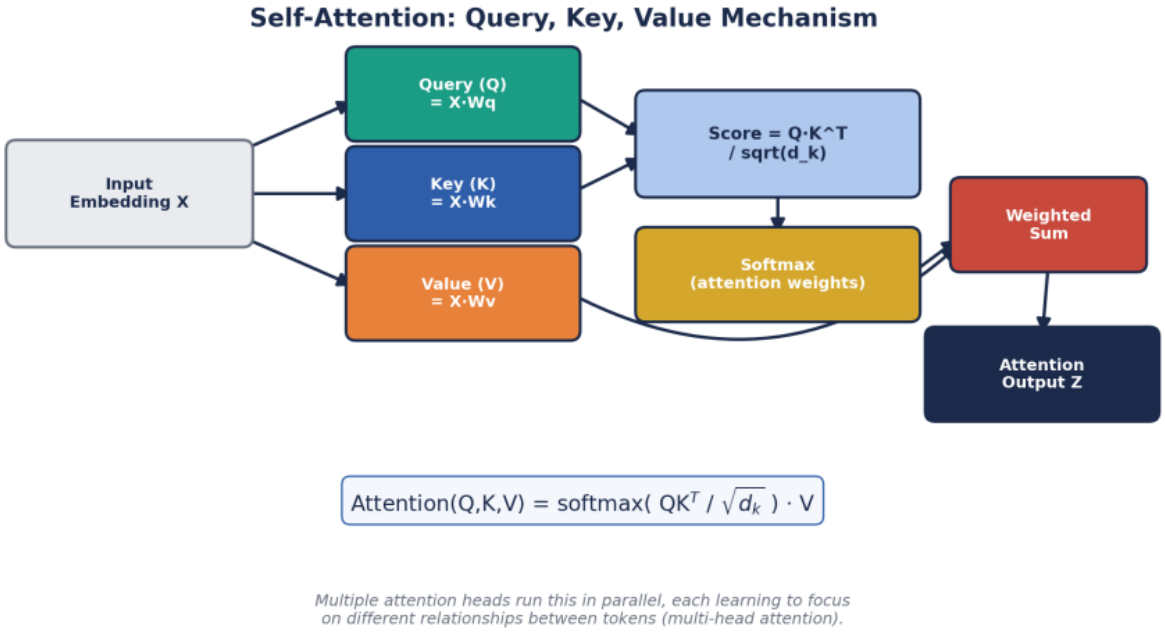


Figure 2.2 — Computing self-attention: Q, K, V are linear projections of the input; scaled dot-product scores are normalized with softmax and used to weight the Value vectors.

- **Multi-Head Attention and Positional Encoding:**

A single attention computation can only learn one type of relationship (e.g. "which word does this pronoun refer to"). **Multi-head attention** runs several attention computations in parallel, each with its own learned Q/K/V projections, so different heads can specialize in different linguistic relationships (syntax, coreference, topic). Because self-attention has no inherent notion of word order, **positional encodings** are added to the input embeddings so the model knows where each token sits in the sequence — modern models increasingly use **Rotary Positional Embeddings (RoPE),** which encode position as a rotation applied to Q/K vectors and generalize well to long sequences.

- **Decoder-Only Architectures**

GPT-style LLMs use only the decoder stack with masked (causal) self-attention: token i can attend only to tokens 1...i, never to future tokens. This causal masking is what makes autoregressive, left-to right text generation possible — at inference time the model predicts one token, appends it to the sequence, and repeats.

- **Example 2.1 — Attention Score Calculation:**

Problem: For two tokens with Query q = [1, 0] and Key vectors k1 = [1, 0] and k2 = [0, 1], with d_k = 2, compute the attention weights?

Solution: Raw scores: $q \cdot k1 = 1$, $q \cdot k2 = 0$.

Scale by $1/\sqrt{d_k} = 1/\sqrt{2} \approx 0.707$, giving 0.707 and 0.

Applying SoftMax: $\exp(0.707)=2.03$, $\exp(0)=1.00$, sum=3.03 → $w1 \approx 0.67$, $w2 \approx 0.33$.

Token 1 receives roughly twice the attention of token 2, matching the intuition that q is more "aligned" with k_1.

3. How LLMs Are Trained:

Training a modern instruction-following LLM is a multi-stage pipeline. Each stage uses a different objective and dataset, and each narrows the model's behavior from "predicts plausible text" toward "gives a helpful, honest, harmless response. “

- **Stage 1 — Pre-training:**

The model is trained on a massive, broad corpus (web text, books, code, papers) using a simple self-supervised objective: predict the next token given all previous tokens (causal language modeling). No human labels are required — the text itself supplies the supervision. This stage consumes the overwhelming majority of training compute and is where the model acquires grammar, facts, reasoning patterns, and world knowledge. The result is a **base model**: fluent, but not yet reliably helpful — it will happily continue a question with more questions rather than answering it, because that is what raw text often does.

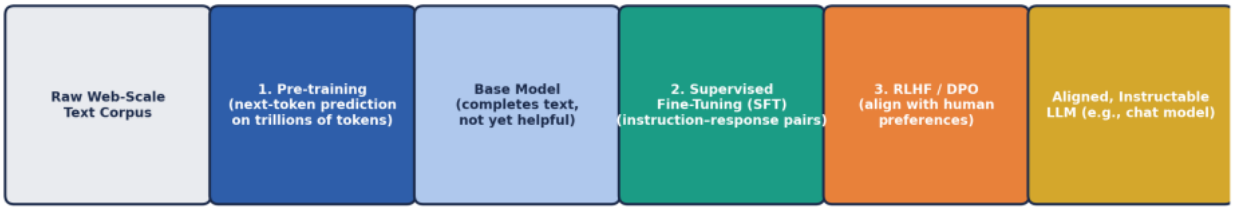


Figure 3.1 — The three-stage LLM training pipeline: pre-training builds broad language competence; supervised fine-tuning teaches response format; RLHF/DPO aligns outputs with human preferences.

- **Stage 2 — Supervised Fine-Tuning (SFT):**

The base model is fine-tuned on a smaller, curated dataset of (instruction, ideal response) pairs, written or vetted by humans. This teaches the model the format of being an assistant — answering questions directly, following instructions, adopting a helpful tone — without yet optimizing for which of several plausible answers humans actually prefer.

- **Stage 3 — RLHF and DPO Alignment:**

Reinforcement Learning from Human Feedback (RLHF) further aligns the model with nuanced human preferences. Human raters compare pairs of model outputs and indicate which is better; this preference data trains a **reward model**, which **then guides reinforcement learning (commonly PPO) to steer the LLM toward higher-reward — i.e. more preferred — responses**.

Direct Preference Optimization (DPO) is **a newer, simpler alternative that optimizes directly on preference pairs without a separate reward model or RL loop, achieving similar alignment with a more stable training process**.

- **Key Distinction:**

Pre-training answers "what is plausible language?" SFT answers "what does a helpful answer look like?" RLHF/DPO answers "which of several helpful answers do humans actually prefer? “All three are necessary; skipping alignment yields a model that is fluent but not trustworthy as an assistant.

- **Example — Pipeline Diagnosis:**

Problem: A team fine-tunes a base LLM only with SFT (no RLHF/DPO) and finds it gives grammatically correct, on-topic, but often unhelpful or verbose answers, and sometimes state

contradictory claims across similar prompts. What stage is missing, and why does that explain the symptoms?

Solution: The missing stage is preference alignment (RLHF or DPO). SFT alone teaches the model to imitate the style of the labeled examples but does not teach it to prefer more useful, concise, or consistent answers over less useful ones — that requires comparative human feedback. Without it, the model can satisfy the surface format of "answering" while still producing responses humans would rank poorly, and small wording changes in the prompt can flip which plausible completion it produces.

4. Major Trends Reshaping LLMs:

Beyond the core architecture and training recipe, four trends currently dominate LLM research and deployment: predictable scaling, retrieval augmentation, sparse (mixture-of experts) architectures, and agentic tool use. Each is a direct response to a limitation of the "vanilla" dense Transformer trained end-to-end.

a) Scaling Laws:

Empirical studies (Kaplan et al. 2020; Hoffmann et al. 2022, "Chinchilla") found that a model's test loss falls as a predictable power law with model size, dataset size, and compute. This turned LLM development from trial-and-error into an engineering discipline: teams can extrapolate from small-scale experiments to decide how many parameters and how much data a target compute budget should be spent on, rather than guessing.

b) Retrieval-Augmented Generation (RAG):

Rather than relying solely on knowledge frozen in the model's parameters at training time, RAG systems retrieve relevant documents from an external knowledge base at inference time and feed them into the prompt as context. This gives the model access to up-to-date, verifiable, and domain-specific information without retraining, and is now the standard architecture for enterprise LLM applications.

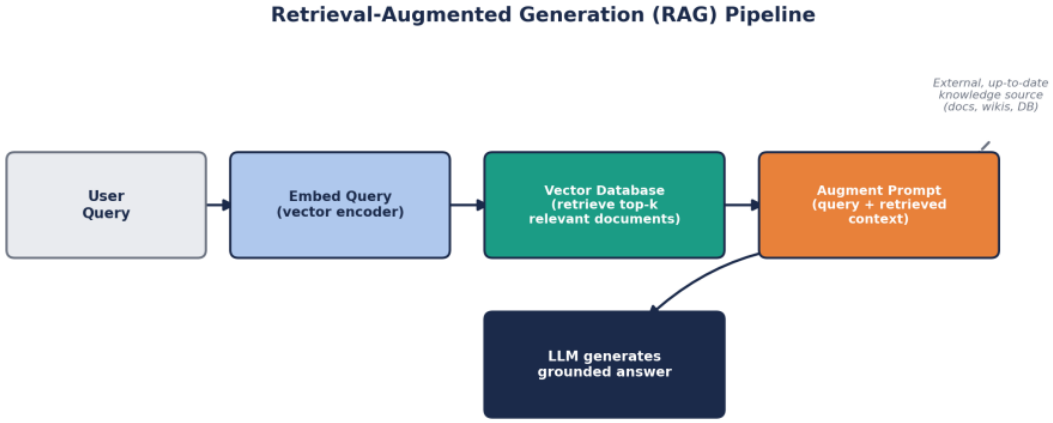


Figure 4.2 — A typical RAG pipeline: the user query is embedded, matched against a vector database, and the retrieved passages are injected into the prompt before generation.

c) Mixture-of-Experts (MoE) Architectures:

MoE models replace a single dense feed-forward block with many parallel "expert" subnetworks and a learned router that activates only a small subset (e.g. 2 of 8) for each token.

This decouples total parameter count (which drives model capacity) from active compute per token (which drives inference cost), letting models like Mixtral and GPT-4-class systems reach very large effective capacity at a fraction of the inference cost of an equivalently sized dense model.

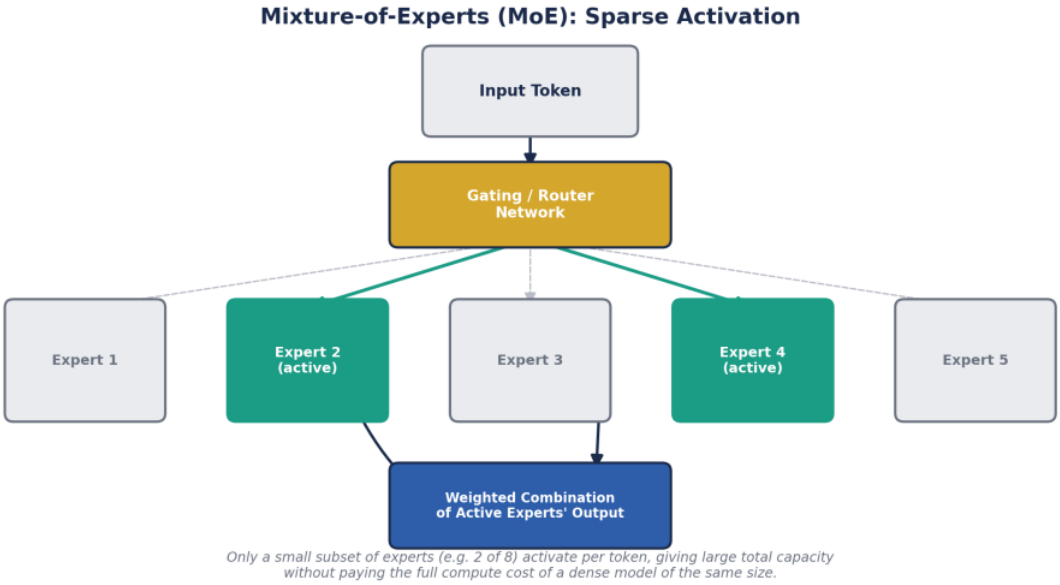


Figure 4.3 — In a Mixture-of-Experts layer, a router selects a small number of experts to process each token, keeping per-token compute low while total model capacity stays high.

d) Agentic and Multimodal LLMs:

The newest trend extends LLMs beyond single-turn text generation into agents that plan, call external tools (search, code execution, APIs), observe results, and iterate toward a goal — and into multimodal models that natively process images, audio, and video alongside text. Sections 6 and 8 return to these trends in depth.

- **Example — Scaling Trade-off:**

Problem: A lab has a fixed compute budget. Should they train a much larger model on the same fixed dataset, or a smaller model on more data, if the goal is the best possible test loss ?

Solution: The Chinchilla scaling laws show that for a fixed compute budget, loss is minimized when model size and training data are scaled together in roughly the same proportion — many earlier models (e.g. GPT-3) were too large relative to the data they were trained on. The correct approach is to jointly increase both, following the compute-optimal ratio, rather than maximizing one dimension while holding the other fixed.